



OC Pizza

HandlePizza

Dossier de conception technique

Version 1.0

Auteur

Antoine de Chaunac
Analyste programmeur



TABLE DES MATIERES

1 - Versions	6
2 - Introduction	7
2.1 - Objet du document	7
2.2 - Références.....	7
3 - Architecture technique.....	8
3.1 - Principes d'architecture et objectifs techniques	8
3.1.1 - Objectifs non fonctionnels (sécurité, performance, traçabilité, évolutivité)	8
3.1.2 - Style d'architecture retenu (monolithe modulaire + API REST).....	8
3.1.3 - Domaines couverts (client, restaurant, général, mobile).....	9
3.2 - Vue globale de la solution	9
3.2.1 - Diagramme de composants (vue globale)	10
3.2.2 - Flux techniques transverses.....	11
3.2.3 - Interactions entre applications	11
3.3 - Socle technique commun	12
3.3.1 - Frontend web	12
3.3.2 - Backend API.....	12
3.3.3 - Sécurité applicative.....	13
3.3.4 - Persistance et accès aux données.....	13
3.3.5 - Intégrations externes.....	13
3.3.6 - Documentation API.....	14
3.3.7 - Observabilité (logs, métriques, traces).....	14
3.4 - Composants métiers backend.....	14
3.4.1 - Authentification et contrôle d'accès.....	15
3.4.2 - Service commande.....	15
3.4.3 - Service paiement.....	15
3.4.4 - Service stock et disponibilité.....	16
3.4.5 - Service réapprovisionnement.....	16
3.4.6 - Service référentiels.....	16
3.4.7 - Service reporting.....	16
3.4.8 - Journalisation et traçabilité métier.....	17
3.5 - Principes techniques des API.....	17
3.5.1 - Conventions REST/JSON et versionning.....	17
3.5.2 - Sécurité JWT et autorisation RBAC.....	17
3.5.3 - Gestion homogène des erreurs	18
3.5.4 - Idempotence, timeouts, retry (intégrations).....	18
3.5.5 - Qualité API et stratégie de tests.....	19
3.6 - Spécificités par interface	19
3.6.1 - Web client	19



3.6.2 - Web restaurant	19
3.6.3 - Web générale.....	20
3.6.4 - Application mobile (Android/iOS)	20
4 - Architecture de déploiement.....	22
4.1 - Vue de déploiement cible	22
4.1.1 - Principes de déploiement (séparation réseau, sécurité, scalabilité)	22
4.1.2 - Diagramme UML de déploiement (vue globale).....	23
4.1.3 - Flux réseau clés (HTTPS, JDBC, messaging)	23
4.2 - Environnements.....	24
4.2.1 - Environnement de développement (DEV).....	24
4.2.2 - Environnement de test/recette (TEST/UAT).....	25
4.2.3 - Environnement de production (PROD).....	25
4.2.4 - Stratégie de promotion entre environnements	25
4.3 - Couche d'exposition web	26
4.3.1 - ALB + ACM (TLS).....	26
4.3.2 - CloudFront + S3 (assets front).....	26
4.3.3 - Règles de routage vers API.....	27
4.4 - Couche applicative backend	28
4.4.1 - Conteneurs Spring Boot (ECS Fargate ou EC2)	28
4.4.2 - Auto scaling horizontal	28
4.4.3 - Health checks et redémarrage	29
4.5 - Couche données et messaging.....	29
4.5.1 - RDS MySQL (sauvegardes, restauration).....	29
4.5.2 - ElastiCache Redis	30
4.5.3 - RabbitMQ (ou service équivalent managé).....	30
4.5.4 - Migrations Flyway au déploiement.....	30
4.6 - Sécurité d'infrastructure	31
4.6.1 - VPC, subnets publics/privés, security groups	31
4.6.2 - Secrets Manager (secrets hors code).....	31
4.6.3 - Chiffrement en transit/au repos	32
4.6.4 - Contrôle d'accès IAM	32
4.7 - Observabilité et exploitation	33
4.7.1 - Logs centralisés (CloudWatch + Logback JSON)	33
4.7.2 - Métriques/alertes (Prometheus/Grafana ou équivalent cloud)	33
4.7.3 - Traces (option v2 OpenTelemetry/Jaeger).....	34
4.7.4 - Runbook incident et supervision.....	34
4.8 - CI/CD et stratégie de livraison	34
4.8.1 - Pipeline GitHub Actions.....	34
4.8.2 - Build/test/scan/deploy	35
4.8.3 - Stratégie de rollback.....	35
4.8.4 - Versionning applicatif et DB	35



4.9 - Intégrations externes en production	36
4.9.1 - Système bancaire (timeouts/retry/circuit breaker).....	36
4.9.2 - Plateforme fournisseur (idempotence/reprise).....	36
4.9.3 - Supervision des appels sortants	37
5 - Architecture logicielle.....	38
5.1 - Principes généraux.....	38
5.1.1 - Style retenu (monolithe modulaire).....	38
5.1.2 - Séparation des responsabilités.....	38
5.1.3 - Conventions de codage et de nommage.....	39
5.2 - Architecture backend	39
5.2.1 - Couches logicielles (Controller, Service, Repository, Domain, Security).....	39
5.2.2 - Modules métier (auth, commande, paiement, stock, réappro, référentiels, reporting, notification, audit).....	40
5.2.3 - Objets d'échange (DTO, mapping MapStruct, validation Bean Validation).....	40
5.2.4 - Transactions et gestion des erreurs.....	40
5.2.5 - Structure des packages/sources backend.....	41
5.3 - Architecture frontend web.....	41
5.3.1 - Organisation SPA (pages, components, routes, state/query)	41
5.3.2 - Gestion des appels API et erreurs UI	42
5.3.3 - Structure des sources frontend.....	42
5.4 - Architecture mobile (Android/iOS).....	43
5.4.1 - Architecture côté mobile (couche UI / domaine / data)	43
5.4.2 - Client API, auth JWT, gestion offline/réseau variable.....	43
5.4.3 - Structure des sources mobile	43
5.5 - Composants transverses	44
5.5.1 - Sécurité applicative.....	44
5.5.2 - Observabilité applicative (logs, métriques, traces).....	44
5.5.3 - Configuration par environnement.....	45
5.6 - Règles d'évolution de l'architecture	45
5.6.1 - Ajout d'un module métier	45
5.6.2 - Comptabilité ascendante API	46
5.6.3 - Critères de refactoring / extraction future.....	46
6 - Points particuliers	47
6.1 - Règles d'évolution de l'architecture	47
6.1.1 - Politique de logs (niveaux, format JSON, corrélation).....	47
6.1.2 - Rétention et exploitation (CloudWatch, recherche incident).....	47
6.1.3 - Données sensibles et masquage	48
6.2 - Configuration et secrets.....	48
6.2.1 - Paramètres par environnements (DEV/TEST/PROD).....	48
6.2.2 - Datasources (RDS, pool, timeouts).....	49
6.2.3 - Secrets Manager (rotation, accès).....	49



6.2.4 - Feature flags / paramètres métiers	50
6.3 - Ressources et performances	50
6.3.1 - Dimensionnement cible (API, DB, cache, MQ)	50
6.3.2 - Limites techniques connues.....	51
6.3.3 - Objectifs de performance (latence, débit, charge).....	51
6.4 - Environnement de développement	52
6.4.1 - Prérequis poste de développement.....	52
6.4.2 - Lancement local (backend, frontend, DB, MQ)	52
6.4.3 - Qualité locale (tests, lint, format).....	53
6.5 - Packaging, livraison et déploiement	53
6.5.1 - Build artefacts (backend/frontend/mobile).....	53
6.5.2 - Pipeline CI/CD (étapes et controles).....	53
6.5.3 - Stratégie de rollback.....	54
6.5.4 - Versionnage app + DB (Flyway).....	54
6.6 - Exploitation et continuité.....	54
6.6.1 - Supervision & alerting.....	55
6.6.2 - Sauvegardes/restauration	55
6.6.3 - Runbook incident.....	55
6.6.4 - Plan d'évolution (v2)	56
7 - Glossaire	57



1 - VERSIONS

Auteur	Date	Description	Version
ADC	05/05/2026	Création du document	0.1
ADC	18/05/2026	Version 1 du document	1.0



2 - INTRODUCTION

2.1 - Objet du document

Le présent document constitue le dossier de conception technique de l'application **HandlePizza**.

Il décrit les choix techniques retenus pour la réalisation du système d'information de **OC Pizza**, en cohérence avec les besoins métiers et le dossier de conception fonctionnelle.

Ce dossier a pour objectif de préciser l'architecture cible, la structuration des composants, les principes d'échange entre interfaces, le modèle de données de référence, ainsi que les exigences transverses (sécurité, performance, traçabilité, maintenabilité).

Il sert de référence pour les équipes de développement, de maintenance et pour l'équipe technique du client.

Les éléments du présent dossier découlent :

- Du cahier des charges et des besoins exprimés par le client OC Pizza
- Du dossier de conception fonctionnelle validé (périmètre, cas d'utilisation, règles de gestion, workflows)
- Des modèles UML et du modèle de données construits lors des travaux de conception
- Des contraintes d'exploitation et d'évolutivité liées à l'extension du réseau de points de vente

2.2 - Références

Pour de plus amples informations, se référer également aux éléments suivants :

1. **DCF – Dossier_conception_fonctionnelle** : Dossier de conception fonctionnelle de l'application
2. ...



3 - ARCHITECTURE TECHNIQUE

3.1 - Principes d'architecture et objectifs techniques

L'architecture technique de **HandlePizza** est conçue pour répondre aux besoins opérationnels du réseau **OC Pizza** tout en restant robuste, maintenable et évolutive.

Elle s'appuie sur une séparation claire des responsabilités entre couche d'accès web, couche applicative, couche de données et intégrations externes.

Les choix techniques retenus visent à garantir la continuité du service en point de vente, la cohérence des traitements métier entre interfaces, la sécurité des échanges et la capacité d'accompagner la croissance du réseau.

3.1.1 - Objectifs non fonctionnels (sécurité, performance, traçabilité, évolutivité)

Les objectifs non fonctionnels prioritaires sont les suivants :

- **Sécurité** : authentification centralisée, contrôle d'accès par rôles et périmètres, chiffrement des échanges HTTPS, protection des opérations sensibles (paiement, réapprovisionnement, pilotage).
- **Performance** : temps de réponse court sur les parcours critiques (prise de commande, validation panier, mise à jour de statut), et maintien d'une expérience fluide en contexte de forte activité.
- **Traçabilité** : journalisation horodatée des actions métier et techniques (commandes, paiements, transitions de statuts, opérations de stock), avec corrélation par identifiants fonctionnels.
- **Évolutivité** : capacité à intégrer de nouveaux points de vente, à augmenter la charge applicative et à étendre les fonctionnalités sans remise en cause de l'architecture globale.
- **Maintenabilité** : organisation modulaire des services, conventions techniques homogènes et documentation API centralisée pour faciliter les évolutions.

3.1.2 - Style d'architecture retenu (monolithe modulaire + API REST)

Le style d'architecture retenu est un monolithe modulaire exposant des **API REST**.



Ce choix permet de limiter la complexité d'exploitation tout en conservant une séparation nette des domaines métier (commande, paiement, stock, réapprovisionnement, référentiels, reporting, traçabilité).

Les modules partagent une base de code et un socle technique commun, mais restent découplés sur le plan fonctionnel via des contrats API internes/externes explicites.

Ce modèle offre un bon compromis entre rapidité de mise en œuvre, cohérence transactionnelle et capacité d'évolution progressive vers une distribution plus fine si la montée en charge l'exige.

3.1.3 - Domaines couverts (*client, restaurant, général, mobile*)

L'architecture couvre quatre domaines applicatifs interconnectés :

- **Domaine client** : parcours digital de commande (consultation offre, panier, validation, paiement en ligne, suivi de statut).
- **Domaine restaurant** : exécution opérationnelle en point de vente (prise de commande comptoir/téléphone, préparation, retrait/livraison, encaissement différé).
- **Domaine général** : pilotage et administration (catalogue, stocks et seuils, réapprovisionnement fournisseur, statistiques d'activité, avis clients).
- **Domaine mobile** : accès nomade aux fonctions pertinentes selon profil (client mobile, usage livreur), avec cohérence des règles métier et des statuts avec les interfaces web.

Ces domaines partagent les mêmes objets métier structurants (commande, paiement, stock, référentiels) afin de garantir une cohérence fonctionnelle transverse à l'échelle du système **HandlePizza**.

3.2 - Vue globale de la solution

La solution **HandlePizza** repose sur une architecture unifiée dans laquelle les applications web client, restaurant, générale et mobile consomment une même couche de services backend.

Le point d'entrée technique est assuré par une couche **Edge Web (reverse proxy)**, qui route les requêtes vers les API métier.

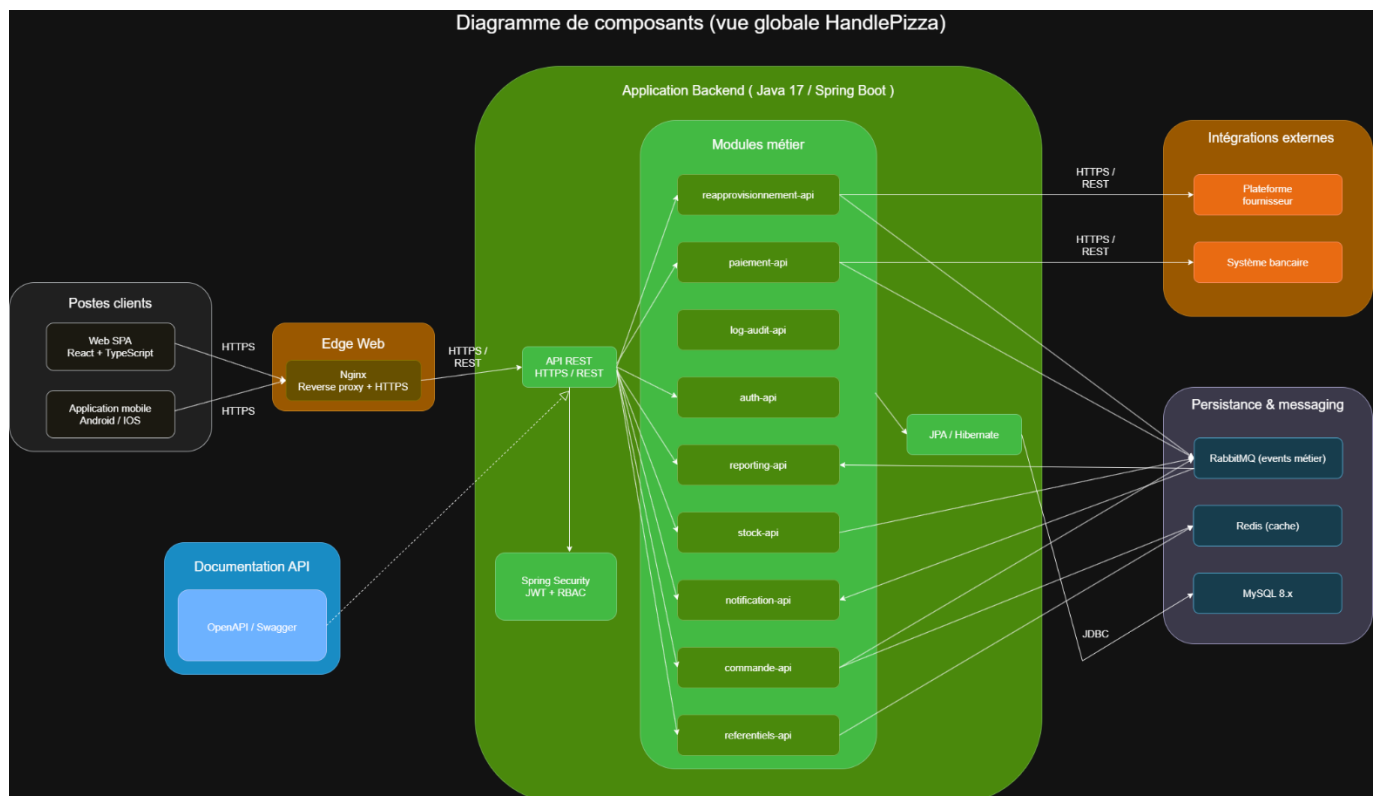
La persistance s'appuie sur une base relationnelle commune, et les intégrations externes sont gérées par des composants dédiés (paiement bancaire, fournisseur).



Cette organisation garantit une cohérence fonctionnelle transverse, une sécurité centralisée et une meilleure maîtrise de l'exploitation.

L'architecture est conçue pour être cloud-compatible ; les choix d'hébergement, de scalabilité et de services d'infrastructure sont détaillés dans la section 4 (Architecture de déploiement).

3.2.1 - Diagramme de composants (vue globale)



Le diagramme de composants global représente les blocs techniques majeurs de la solution et leurs dépendances :

- **Postes clients** : navigateur web et applications mobiles (Android/iOS).
- **Edge Web** : Nginx en point d'entrée **HTTPS** et **reverse proxy**.
- **Application Backend** : **Spring Boot API**, sécurité (**JWT + rôles**), accès aux données (**JPA/Hibernate**).
- **Persistance** : **MySQL 8.x** (base relationnelle métier), **Redis** (cache) et **RabbitMQ** (événements métier).
- **Intégrations externes** : système bancaire et plateforme fournisseur.
- **Documentation** : **OpenAPI/Swagger** pour le contrat d'API.



Le diagramme associé fournit une lecture d'ensemble de la structure technique cible, sans entrer au niveau détaillé des implémentations internes.

3.2.2 - Flux techniques transverses

Les flux techniques transverses principaux sont les suivants :

- Frontends web/mobile → point d'entrée **Nginx via HTTPS**.
- **Nginx** → **backend API** via routage interne sécurisé.
- **Backend API** ↔ base de données **MySQL via JDBC/JPA**.
- API Paiement ↔ système bancaire via **HTTPS/REST**.
- API Réapprovisionnement ↔ plateforme fournisseur via **HTTPS/REST**.
- API ↔ composants de journalisation/traçabilité pour supervision et audit.
- Documentation **OpenAPI/Swagger** ↔ API pour publication des contrats techniques.

Ces flux assurent la continuité de traitement métier, la sécurisation des échanges et la cohérence des statuts entre canaux.

3.2.3 - Interactions entre applications

Les applications interagissent via les services backend partagés, selon leur périmètre métier :

- **Application web client** : consomme principalement les services commande, paiement, référentiels et suivi de statut.
- **Application web restaurant** : consomme les services commande, paiement, stock et distribution (retrait/livraison).
- **Application web générale** : consomme les services référentiels, stock, réapprovisionnement et reporting.
- **Application mobile** : consomme les API nécessaires aux parcours client mobile et aux usages terrain (notamment livraison).

Les interactions sont pilotées par des règles métier communes afin de garantir :

- L'unicité des états de commande,
- La cohérence des données (paiement, stock, statuts),
- La synchronisation en temps réel entre les interfaces.



3.3 - Socle technique commun

Le socle technique commun de **HandlePizza** fournit une base homogène aux interfaces web client, web restaurant, web générale et mobile.

Le canal mobile s'appuie sur des implémentations natives : **Kotlin/Jetpack Compose** pour Android et **Swift/SwiftUI** pour iOS, en cohérence avec les principes **API REST** sécurisés du socle commun.

Il garantit la cohérence des règles métier, la standardisation des échanges, la sécurisation des accès et la fiabilité des traitements transverses.

3.3.1 - Frontend web

Le frontend web repose sur une **architecture SPA en HTML5/CSS3/TypeScript avec React**, offrant une interface responsive adaptée aux usages sur poste fixe et mobile.

La chaîne frontend s'appuie sur **Vite** (build/dev server), **React Router** (navigation), **TanStack Query** (gestion des appels API et cache client), et **React Hook Form + Zod** (gestion/validation des formulaires).

Le temps réel de suivi des statuts de commande est assuré via **WebSocket (STOMP)** ou **SSE** selon le canal.

Ce socle permet :

- Une navigation fluide sans rechargement complet de page ;
- Une consommation standardisée des **endpoints API REST** ;
- Une gestion claire des états d'interface (panier, commande, statut) ;
- Une meilleure maintenabilité par composants réutilisables.

3.3.2 - Backend API

Le backend est implémenté en **Java 17 + Spring Boot 3**, sous forme de monolithe modulaire exposant des **API REST JSON**.

Il centralise les règles métier et les services transverses : commande, paiement, stock, réapprovisionnement, référentiels, reporting, audit.



Le socle backend intègre également **Bean Validation** (validation des entrées), **MapStruct** (mapping DTO/entités) et **Spring Scheduler** (tâches planifiées).

Ce socle garantit :

- Cohérence des traitements entre interfaces ;
- Organisation modulaire des responsabilités ;
- Evolutivité fonctionnelle sans rupture d'architecture.

3.3.3 - Sécurité applicative

La sécurité applicative s'appuie sur **Spring Security avec authentification par JWT et autorisation RBAC (rôles)**.

Le contrôle d'accès est appliqué selon le profil utilisateur et le périmètre métier (point de vente / réseau).

Les échanges sont protégés par **HTTPS** et les opérations sensibles (paiement, réapprovisionnement, pilotage) sont soumises à des contrôles renforcés.

3.3.4 - Persistance et accès aux données

La persistance relationnelle repose sur **MySQL 8.x**.

L'accès aux données est réalisé via **JPA / Hibernate (mécanisme JDBC sous-jacent)** afin d'assurer la cohérence objet-relationnel et la fiabilité transactionnelle.

Les migrations de schéma sont versionnées et automatisées via **Flyway**.

Des composants complémentaires sont utilisés :

- **Redis** pour le cache applicatif (accès rapides sur données fréquentes) ;
- **RabbitMQ** pour les échanges asynchrones et la diffusion d'événements métier.

3.3.5 - Intégrations externes

Les intégrations externes sont exposées via des connecteurs API dédiés :

- Système bancaire pour l'autorisation/refus des paiements ;



- Plateforme fournisseur pour le cycle de réapprovisionnement.

Les échanges se font en **HTTPS/REST**, avec gestion des erreurs réseau, timeouts et reprise contrôlée.

La résilience des appels externes s'appuie sur **Resilience4j (retry, circuit breaker, bulkhead)**.

3.3.6 - Documentation API

La documentation des services est centralisée via **OpenAPI / Swagger**.

Elle formalise le contrat technique des **endpoints** (entrées, sorties, statuts, erreurs), facilite la recette technique et améliore la communication entre équipes (développement, test, exploitation).

3.3.7 - Observabilité (logs, métriques, traces)

L'observabilité repose sur trois axes complémentaires :

- Logs : journalisation structurée des événements techniques et métier (**Logback JSON**) ;
- Métriques : suivi des performances applicatives (latence, erreurs, charge) via **Micrometer + Prometheus** ;
- Traces : suivi de bout en bout des parcours critiques (commande, paiement, livraison).

La visualisation et l'alerte sont assurées via **Grafana**.

En évolution cible (v2), la traçabilité distribuée pourra être renforcée via **OpenTelemetry + Jaeger**.

Ce socle permet une supervision proactive, une détection rapide des incidents et une meilleure capacité de diagnostic.

3.4 - Composants métiers backend

Les composants métiers backend implémentent les règles fonctionnelles centrales de **HandlePizza**.



Ils sont organisés par domaines afin d'assurer une séparation claire des responsabilités, une meilleure maintenabilité et une cohérence de traitement entre les interfaces client, restaurant, générale et mobile.

3.4.1 - Authentification et contrôle d'accès

Ce composant gère l'identification des utilisateurs et l'application des autorisations.

Fonctions principales :

- Authentification par credentials et émission de **token JWT**,
- Contrôle d'accès **RBAC** selon les rôles (client, employé, pizzaiolo, livreur, manager, direction),
- Contrôle de périmètre (point de vente local / périmètre réseau),
- Sécurisation des **endpoints** sensibles.

3.4.2 - Service commande

Ce service pilote le cycle de vie complet d'une commande.

Fonctions principales :

- Création, validation, modification/annulation selon règles métier,
- Gestion des statuts de commande (confirmed, in_preparation, ready_to_pickup, in_delivery, delivered, cancelled),
- Orchestration entre parcours client et exécution restaurant,
- Publication des mises à jour de statut pour le suivi temps réel.

3.4.3 - Service paiement

Ce service gère les règlements en ligne et différés.

Fonctions principales :

- Initialisation des transactions de paiement,
- Communication avec le système bancaire pour autorisation/refus,
- Mise à jour des statuts de paiement (in_progress, validate, cancelled),
- Gestion des cas d'échec/réessai et traçabilité des opérations d'encaissement.



3.4.4 - Service stock et disponibilité

Ce service maintient la cohérence entre ventes, recettes et niveaux de stock.

Fonctions principales :

- Mise à jour des quantités d'ingrédients par point de vente,
- Gestion des seuils de réapprovisionnement,
- Recalcul de la disponibilité produit en fonction des recettes,
- Prévention de la vente de produits non réalisables.

3.4.5 - Service réapprovisionnement

Ce service gère le cycle de commande fournisseur.

Fonctions principales :

- Création et suivi des commandes de réapprovisionnement,
- Gestion des statuts (launched, confirmed, in_progress, completed),
- Intégration des informations de réception (complète/partielle),
- Synchronisation avec le service stock pour mise à jour des quantités.

3.4.6 - Service référentiels

Ce service centralise les données de référence partagées par l'ensemble du système.

Fonctions principales :

- Gestion des référentiels produits (pizzas, ingrédients, unités),
- Gestion des comptes, rôles et points de vente,
- Gestion des statuts métiers de référence,
- Mise à disposition de données cohérentes pour les autres services.

3.4.7 - Service reporting

Ce service fournit les données de pilotage opérationnel et décisionnel.

Fonctions principales :



- Consolidation des indicateurs d'activité (commandes, paiement, délais, volumétrie),
- Production de vues de suivi local (manager) et global (direction),
- Exposition d'**API** de consultation pour tableaux de bord,
- Support au suivi de performance par point de vente et à l'échelle réseau.

3.4.8 - Journalisation et traçabilité métier

Ce composant assure la traçabilité des opérations critiques.

Fonctions principales :

- Journalisation horodatée des actions métier (statuts commande, paiement, stock, réapprovisionnement),
- Corrélation des événements via identifiants fonctionnels (commande, utilisateur, point de vente),
- Support audit et diagnostic incident,
- Alimentation des besoins de supervision et de conformité interne.

3.5 - Principes techniques des API

Les API de **HandlePizza** suivent des conventions techniques communes afin de garantir la cohérence des échanges, la sécurité des accès, la robustesse des intégrations et la maintenabilité globale de la solution.

3.5.1 - Conventions REST/JSON et versionning

Les services sont exposés en **API REST** avec des messages au format **JSON**.

Principes retenus :

- **Endpoints versionnés (ex : /api/v1/...)** pour préserver la compatibilité évolutive ;
- Usage standard des verbes **HTTP (GET, POST, PUT/PATCH, DELETE)** ;
- Nomenclature homogène des ressources (noms explicites, conventions stables) ;
- Séparation claire entre objets d'entrée/sortie API et modèle de persistance interne ;
- Codes **HTTP** cohérents selon le résultat métier et technique.

3.5.2 - Sécurité JWT et autorisation RBAC



L'accès aux API protégées repose sur un **token Bearer JWT** et une autorisation par rôles.

Principes retenus :

- Authentification centralisée et émission de **JWT signés** (access token + refresh token) ;
- Contrôle d'accès **RBAC** selon le profil (client, employé, pizzaiolo, livreur, manager, direction) ;
- Restrictions de périmètre (point de vente local vs réseau) sur les endpoints concernés ;
- Filtrage des opérations sensibles (paiement, réapprovisionnement, pilotage) ;
- Echanges API exclusivement en **HTTPS**.

3.5.3 - Gestion homogène des erreurs

Les **erreurs API** suivent un format de réponse standardisé pour simplifier le traitement côté frontend/mobile et le diagnostic.

Principes retenus :

- Structure homogène des erreurs (code, message, détails, horodatage, identifiant de corrélation) ;
- Mapping explicite des erreurs métier et techniques vers les **statuts HTTP** ;
- Messages fonctionnels clairs pour les cas utilisateurs (ex: paiement refusé, produit indisponible) ;
- Journalisation systématique des erreurs critiques pour audit et support.

3.5.4 - Idempotence, timeouts, retry (intégrations)

Les appels vers les systèmes externes (bancaire, fournisseur) sont encadrés pour limiter les effets de bord en cas d'aléa réseau.

Principes retenus :

- Idempotence des opérations sensibles lorsque nécessaire (éviter les doublons de traitement) ;
- Timeouts explicites sur les appels sortants ;
- Stratégie de retry contrôlé sur erreurs transitoires ;
- Arrêt de propagation des pannes externes via mécanismes de protection (**Resilience4j : retry, circuit breaker, bulkhead**) ;
- Traçabilité des tentatives et des résultats d'intégration.



3.5.5 - Qualité API et stratégie de tests

La qualité des API est assurée par une stratégie de tests multi-niveaux couvrant la logique métier, l'intégration technique et le contrat d'interface.

Principes retenus :

- Tests unitaires : validation des règles métier et composants applicatifs via **JUnit 5 et Mockito**.
- Tests d'intégration : vérification des interactions avec les dépendances techniques (base de données, messaging, configuration) via **Testcontainers**.
- Tests d'API : validation des **endpoints REST (statuts HTTP, formats JSON, cas nominal/erreur, sécurité)** via **RestAssured** et/ou collections **Postman**.
- Vérification du contrat API : alignement continu entre implémentation et documentation **OpenAPI/Swagger**.
- Automatisation : exécution des tests dans le **pipeline CI** afin de détecter rapidement les régressions avant livraison.

3.6 - Spécificités par interface

Chaque interface de **HandlePizza** partage le même socle technique, mais applique des priorités fonctionnelles et techniques propres à son contexte d'usage.

3.6.1 - Web client

L'interface web client est orientée parcours de commande digital et expérience utilisateur fluide.

Spécificités principales :

- Consultation du catalogue filtré par point de vente et disponibilité réelle ;
- Gestion du panier et validation de commande (retrait/livraison) ;
- Paiement en ligne et suivi de commande en temps réel ;
- Possibilité de modification/annulation selon règles métier avant préparation ;
- Exigence forte de lisibilité des erreurs fonctionnelles (paiement refusé, produit indisponible, etc.).

3.6.2 - Web restaurant

DCAC

DCAC.COM

100 boulevard de Paris – 01 23 45 67 89 10 – DCACemail@dcac.com

S.A.R.L. au capital de 1 000,00 € enregistrée au RCS de xxxx – SIREN 999 999 999 – Code APE : 6202A



L'interface web restaurant est orientée exécution opérationnelle en point de vente.

Spécificités principales :

- Prise de commande comptoir/téléphone ;
- Pilotage de la préparation (prise en charge, progression, fin de préparation) ;
- Gestion distribution retrait/livraison ;
- Encaissement différé au comptoir ou à la livraison ;
- Mise à jour rapide et fiable des statuts en contexte de rush ;
- Traçabilité stricte des actions par rôle (employé, pizzaiolo, livreur).

3.6.3 - Web générale

L'interface web générale est orientée pilotage, supervision et administration transverse.

Spécificités principales :

- Gestion du catalogue produits et des référentiels ;
- Supervision des stocks et seuils de réapprovisionnement ;
- Suivi des commandes fournisseur et réceptions ;
- Consultation des statistiques d'activité et des avis clients ;
- Contrôle d'accès renforcé selon périmètre (manager local vs direction réseau) ;
- Cohérence des données partagées avec les interfaces client et restaurant.

3.6.4 - Application mobile (Android/iOS)

L'application mobile couvre les usages nomades, avec priorité aux actions rapides et au contexte réseau variable.

Spécificités principales :

- Implémentations natives : Android en **Kotlin + Jetpack Compose (Retrofit/OkHttp)** et iOS en **Swift + SwiftUI (URLSession/Alamofire)** ;
- Parcours client mobile (consultation, commande, suivi) ;
- Usages opérationnels terrain selon profil (notamment livreur) ;
- Consommation sécurisée des **API REST via HTTPS et token JWT (Bearer)** ;
- Gestion des aléas réseau (timeouts, retry, synchronisation à reconnexion, refresh token) ;
- Ergonomie optimisée pour interactions courtes et lecture immédiate ;



- Cohérence stricte des règles métier et statuts avec les interfaces web.



4 - ARCHITECTURE DE DEPLOIEMENT

La vue de déploiement cible de **HandlePizza** décrit l'organisation technique de l'hébergement de la solution, la répartition des composants par couche et les flux réseau critiques.

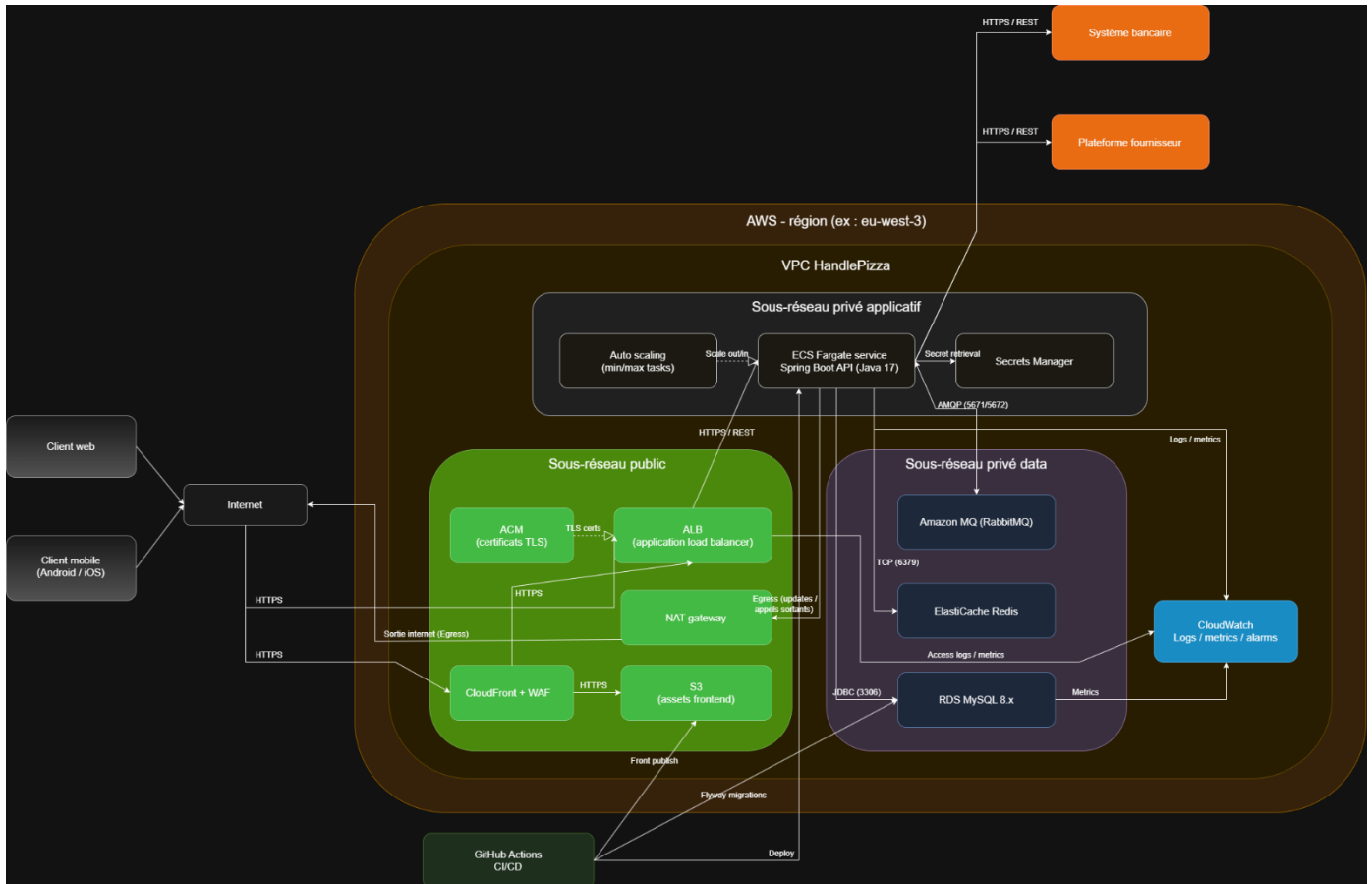
Elle vise à garantir la sécurité des accès, la robustesse opérationnelle et la capacité de montée en charge, tout en conservant une architecture exploitable à l'échelle du réseau **OC Pizza**.

4.1 - Vue de déploiement cible

4.1.1 - Principes de déploiement (séparation réseau, sécurité, scalabilité)

Le déploiement cible repose sur les principes suivants :

- **Séparation réseau** : isolation des couches d'exposition, applicative et données, avec accès contrôlés entre zones.
- **Sécurité par défaut** : exposition publique limitée au point d'entrée HTTPS ; services internes (API, base, cache, messaging) non exposés directement à Internet.
- **Segmentation des responsabilités** : routage web, traitements métier, persistance et intégrations externes séparés en composants dédiés.
- **Scalabilité horizontale** : possibilité d'augmenter le nombre d'instances backend selon la charge (pics d'activité, rush horaires).
- **Résilience opérationnelle** : supervision continue, redémarrage contrôlé des services, sauvegardes régulières et capacité de reprise.
- **Cohérence inter-environnements** : mêmes principes techniques appliqués entre DEV, TEST/UAT et PROD, avec paramétrages adaptés.





Les flux réseau clés de la cible sont les suivants :

- Clients web/mobile → point d'entrée : **HTTPS (TLS)** ;
- Point d'entrée → backend API : **HTTPS/REST** ;
- Backend API → base de données **MySQL : JDBC** (réseau privé) ;
- Backend API → **Redis** : protocole cache interne (réseau privé) ;
- Backend API ↔ **RabbitMQ** : échanges asynchrones d'événements métier ;
- Backend API ↔ système bancaire : **HTTPS/REST** (paiement) ;
- Backend API ↔ plateforme fournisseur : **HTTPS/REST** (réapprovisionnement) ;
- Backend/API/infra → observabilité : flux logs, métriques et traces pour supervision.

Ces flux sont filtrés par règles d'accès réseau strictes afin de limiter la surface d'exposition et de préserver la confidentialité, l'intégrité et la disponibilité du système.

Les sections suivantes détaillent, pour chaque couche technique, le rôle des services retenus, leurs règles de configuration cible et les contraintes d'exploitation associées.

4.2 - Environnements

Le déploiement de **HandlePizza** s'appuie sur une séparation stricte des environnements afin de sécuriser les cycles de livraison, fiabiliser les validations et limiter les risques de régression en production.

Chaque environnement reprend la même architecture logique, avec un niveau de capacité, de sécurité et de supervision adapté à son usage.

4.2.1 - Environnement de développement (DEV)

L'environnement **DEV** est dédié aux développements fonctionnels et techniques en cours.

Caractéristiques principales :

- Déploiements fréquents et itératifs ;
- Données de test non sensibles ;
- Services cloud dimensionnés de manière économique ;
- Journalisation et métriques orientées diagnostic rapide ;
- Exécution systématique des tests unitaires et tests techniques de base.

Objectif : permettre un cycle rapide de développement et de correction.



4.2.2 - Environnement de test/recette (TEST/UAT)

L'environnement **TEST/UAT** est destiné à la validation fonctionnelle, technique et métier avant mise en production.

Caractéristiques principales :

- Architecture proche de la production ;
- Jeux de données de recette représentatifs (anonymisés si nécessaire) ;
- Tests d'intégration, tests d'API et scénarios de non-régression ;
- Validation des flux externes (paiement/fournisseur) sur environnements de test partenaires ;
- Contrôle des performances sur parcours critiques.

Objectif : valider la conformité de la version candidate avant promotion en **PROD**.

4.2.3 - Environnement de production (PROD)

L'environnement **PROD** héberge le service opérationnel utilisé par les clients et les équipes **OC Pizza**.

Caractéristiques principales :

- Haute disponibilité et scalabilité configurées ;
- Politiques de sécurité renforcées (accès, secrets, chiffrement, segmentation réseau) ;
- Supervision continue (logs, métriques, alertes) ;
- Sauvegardes planifiées et procédures de restauration ;
- Processus de déploiement contrôlé avec possibilité de rollback.

Objectif : garantir la continuité de service, la sécurité et la fiabilité des traitements métier.

4.2.4 - Stratégie de promotion entre environnements

La promotion d'une version suit un pipeline contrôlé : **DEV** → **TEST/UAT** → **PROD**.

Principes retenus :

- Validation des tests automatisés à chaque étape (unitaires, intégration, API) ;
- Artefacts versionnés et promotionnés sans reconstruction ;



- Validation métier en **UAT** avant passage en production ;
- Exécution des migrations de base de données via **Flyway** dans un ordre maîtrisé ;
- Stratégie de rollback définie en cas d'incident (application + données compatibles) ;
- Traçabilité complète des livraisons (version, date, auteur, environnement).

Cette stratégie réduit les risques de mise en production et sécurise l'évolution continue de la plateforme **HandlePizza**.

4.3 - Couche d'exposition web

La couche d'exposition web constitue le point d'entrée réseau de **HandlePizza** pour les clients web/mobile.

Elle assure la terminaison **TLS**, la distribution des contenus frontend et le routage sécurisé vers la couche applicative backend, conformément au diagramme de déploiement validé.

4.3.1 - ALB + ACM (TLS)

Le trafic **HTTPS** applicatif est reçu par un **Application Load Balancer (ALB)**.

Le chiffrement **TLS** est géré par **AWS Certificate Manager (ACM)**, qui fournit et renouvelle les certificats utilisés en entrée.

Rôle principal :

- Exposition sécurisée des endpoints publics ;
- Terminaison **TLS** au niveau **ALB** ;
- Répartition du trafic vers les services backend **Spring Boot** ;
- Intégration avec les vérifications d'état (**health checks**) pour ne router que vers des cibles saines.

Points techniques clés :

- Protocole d'entrée : **HTTPS** ;
- Certificats gérés centralement via **ACM** ;
- Routage backend uniquement vers la couche applicative privée ;
- Logs d'accès et métriques disponibles pour supervision.

4.3.2 - CloudFront + S3 (assets front)

DCAC

DCAC.COM

100 boulevard de Paris – 01 23 45 67 89 10 – DCACemail@dcac.com

S.A.R.L. au capital de 1 000,00 € enregistrée au RCS de Xxxx – SIREN 999 999 999 – Code APE : 6202A



Les assets statiques frontend (fichiers **HTML/CSS/JS**, médias) sont hébergés dans **Amazon S3** et distribués via **CloudFront**.

La protection en bordure est assurée par **WAF** rattaché à **CloudFront**, conformément au schéma validé (**CloudFront + WAF**).

Rôle principal :

- Diffusion performante des ressources statiques ;
- Réduction de latence côté utilisateur via **cache edge** ;
- Protection en amont contre certains trafics malveillants ;
- Séparation claire entre distribution frontend statique et trafic API dynamique.

Points techniques clés :

- Accès client en **HTTPS** ;
- **CloudFront** sert les assets depuis **S3** ;
- Publication des assets via pipeline **CI/CD** ;
- Politique de cache paramétrée selon type de ressource.

4.3.3 - Règles de routage vers API

Le routage distingue les flux frontend statiques et les flux API métier :

- Requêtes de contenu statique : orientées vers **CloudFront/S3** ;
- Requêtes API : orientées vers **ALB** puis vers le service backend **Spring Boot** en sous-réseau privé applicatif.

Règles retenues :

- Séparation explicite des chemins (assets vs API) ;
- Conservation des en-têtes nécessaires à la sécurité et à la traçabilité ;
- Filtrage et contrôle d'accès dès la couche d'exposition ;
- Terminaison **HTTPS** en entrée et transit sécurisé vers les services applicatifs.

Cette organisation garantit une exposition maîtrisée, des performances front optimisées et un acheminement fiable des appels API vers le backend.



4.4 - Couche applicative backend

La couche applicative backend exécute les services métier de **HandlePizza** (commande, paiement, stock, réapprovisionnement, référentiels, reporting, notification, audit).

Elle est déployée en sous-réseau privé applicatif, sans exposition directe à Internet, et reçoit le trafic uniquement via la couche d'exposition web (**ALB**).

4.4.1 - Conteneurs Spring Boot (ECS Fargate ou EC2)

L'application backend **Spring Boot (Java 17)** est packagée en conteneur et déployée sur une infrastructure de type **ECS Fargate** (cible privilégiée) ou **EC2** selon contraintes d'exploitation.

Principes retenus :

- Exécution isolée par conteneur pour homogénéité des déploiements ;
- Configuration externalisée par environnement (**DEV/TEST/PROD**) ;
- Accès aux secrets via service dédié (**Secrets Manager**) ;
- Communication interne vers **RDS, Redis, RabbitMQ** et intégrations externes ;
- Déploiement piloté par pipeline **CI/CD**.

Ce modèle améliore la portabilité, la reproductibilité des versions et la maintenabilité opérationnelle.

4.4.2 - Auto scaling horizontal

La montée en charge est assurée par un mécanisme d'auto scaling horizontal des tâches backend.

Principes retenus :

- Définition d'un nombre minimal et maximal de tâches ;
- Ajout/retrait automatique d'instances selon la charge (CPU, mémoire, trafic) ;
- Adaptation dynamique aux pics d'activité (rush de commandes) ;
- Maintien de la disponibilité en cas d'augmentation soudaine du trafic.

Ce mécanisme permet d'ajuster la capacité sans intervention manuelle et d'optimiser l'équilibre performance/coût.



4.4.3 - Health checks et redémarrage

La robustesse applicative repose sur des contrôles d'état continus et des mécanismes de reprise automatique.

Principes retenus :

- Endpoints de santé exposés par l'application (ex : /actuator/health) ;
- Vérifications périodiques par la couche de routage/orchestration ;
- Retrait automatique des instances non saines du routage ;
- Redémarrage contrôlé des tâches défaillantes ;
- Supervision des incidents via logs et métriques.

Cette stratégie limite l'impact des pannes locales et garantit un service backend stable en production.

4.5 - Couche données et messaging

La couche données et messaging supporte la persistance métier, les accès rapides en cache et les traitements asynchrones.

Elle est déployée en sous-réseau privé data afin de limiter la surface d'exposition et de renforcer la sécurité des échanges internes.

4.5.1 - RDS MySQL (sauvegardes, restauration)

La base relationnelle principale est hébergée sur **Amazon RDS MySQL 8.x**.

Elle stocke les données métier structurantes : commandes, paiements, stocks, réapprovisionnement, référentiels, audit.

Principes retenus :

- Accès autorisé uniquement depuis la couche applicative privée ;
- Sauvegardes automatiques planifiées avec politique de rétention ;
- Possibilité de restauration à un point cohérent ;
- Supervision des performances et de la disponibilité via métriques ;
- Chiffrement en transit et au repos selon la politique de sécurité.



Objectif : garantir intégrité, durabilité et reprise maîtrisée des données critiques.

4.5.2 - *ElastiCache Redis*

Le cache applicatif est assuré par **ElastiCache Redis**.

Il est utilisé pour réduire la latence sur les données fréquemment consultées et limiter la charge sur la base relationnelle.

Usages principaux :

- Cache de consultation (catalogue, disponibilité produit, référentiels fréquents) ;
- Accélération de réponses sur parcours à forte volumétrie ;
- Support de patterns techniques nécessitant des accès mémoire rapides.

Objectif : améliorer la réactivité des interfaces et la scalabilité globale.

4.5.3 - *RabbitMQ (ou service équivalent managé)*

Les échanges asynchrones reposent sur **RabbitMQ** (via **Amazon MQ** ou équivalent managé).

Ce composant découple les traitements producteurs/consommateurs et fiabilise les flux événementiels.

Usages principaux :

- Diffusion d'événements métier (commande, paiement, stock, notification) ;
- Traitement différé de tâches non bloquantes ;
- Meilleure résilience des traitements en cas de charge variable.

Objectif : fluidifier les parcours temps réel côté utilisateur tout en sécurisant les traitements arrière-plan.

4.5.4 - *Migrations Flyway au déploiement*

La gestion de schéma est pilotée via **Flyway**, exécuté dans la chaîne de déploiement **CI/CD**.

Chaque version applicative embarque les scripts de migration nécessaires à l'évolution de la base.



Principes retenus :

- Versionnement explicite des scripts de migration ;
- Exécution ordonnée et traçable par environnement ;
- Non-régression du modèle de données entre versions ;
- Compatibilité contrôlée entre version applicative et version DB ;
- Stratégie de rollback applicatif/documentée en cas d'échec de migration.

Objectif : sécuriser l'évolution continue du schéma de données sans rupture de service.

4.6 - Sécurité d'infrastructure

La sécurité d'infrastructure de **HandlePizza** repose sur une stratégie de défense en profondeur : segmentation réseau, contrôle strict des accès, protection des secrets et chiffrement systématique des données sensibles.

4.6.1 - VPC, subnets publics/privés, security groups

L'infrastructure est isolée dans un **VPC dédié**, segmenté en sous-réseaux publics et privés :

- Sous-réseau public : composants d'exposition (**CloudFront/ALB, NAT Gateway**).
- Sous-réseau privé applicatif : services backend (**ECS Fargate**).
- Sous-réseau privé data : **RDS MySQL, ElastiCache Redis, RabbitMQ managé**.

Les security groups appliquent le principe du moindre privilège :

- Seuls les flux nécessaires sont autorisés entre couches ;
- Aucune exposition directe Internet des services applicatifs/data ;
- Accès base/cache/messaging limité aux services backend autorisés.

4.6.2 - Secrets Manager (secrets hors code)

Les secrets techniques sont externalisés dans **AWS Secrets Manager** :

- Identifiants base de données ;
- Clés d'accès API externes ;
- Secrets applicatifs sensibles.

Principes retenus :

DCAC

DCAC.COM

100 boulevard de Paris – 01 23 45 67 89 10 – DCACemail@dcac.com

S.A.R.L. au capital de 1 000,00 € enregistrée au RCS de XXXX – SIREN 999 999 999 – Code APE : 6202A



- Aucun secret en dur dans le code, les images ou les dépôts ;
- Récupération dynamique des secrets au runtime ;
- Rotation contrôlée des secrets selon la politique de sécurité ;
- Traçabilité des accès aux secrets.

4.6.3 - Chiffrement en transit/au repos

Les données sont protégées en transit et au repos.

En transit :

- **HTTPS/TLS** pour les flux clients, API et intégrations externes ;
- Canaux sécurisés entre composants internes selon protocole supporté.

Au repos :

- Chiffrement des volumes/données des services managés (**RDS**, cache, messaging, stockage objet) ;
- Protection des sauvegardes selon la même politique de chiffrement.

Objectif : préserver confidentialité et intégrité des données tout au long de leur cycle de vie.

4.6.4 - Contrôle d'accès IAM

Le contrôle d'accès aux ressources cloud est géré via **IAM** avec séparation des rôles techniques et opérationnels.

Principes retenus :

- Permissions minimales par service/compte (least privilege) ;
- Rôles dédiés pour exécution applicative, **CI/CD**, exploitation et administration ;
- Journalisation et audit des actions sensibles ;
- Limitation des accès humains directs aux environnements de production ;
- Gouvernance des accès par revue périodique des droits.

Cette approche réduit le risque d'erreur ou d'accès non autorisé et renforce la sécurité globale de la plateforme.



4.7 - Observabilité et exploitation

L'observabilité et l'exploitation de **HandlePizza** visent à détecter rapidement les incidents, diagnostiquer les causes racines et garantir la continuité de service.

La stratégie repose sur trois piliers complémentaires : logs, métriques/alertes, traces.

4.7.1 - Logs centralisés (CloudWatch + Logback JSON)

Les services backend publient des logs structurés au format **JSON via Logback**.

Les journaux sont centralisés dans **CloudWatch Logs** pour consultation, corrélation et audit.

Principes retenus :

- Format homogène des logs applicatifs et techniques ;
- Horodatage systématique et identifiants de corrélation ;
- Séparation des niveaux (**INFO/WARN/ERROR**) ;
- Conservation selon politique de rétention par environnement ;
- Exploitation des logs pour diagnostic incident et traçabilité métier.

4.7.2 - Métriques/alertes (Prometheus/Grafana ou équivalent cloud)

Les métriques de santé et de performance sont collectées en continu pour suivre le comportement de la plateforme.

Métriques principales :

- Disponibilité des services ;
- Latence des API ;
- Taux d'erreurs HTTP ;
- Charge CPU/mémoire ;
- Volumétrie des flux (commandes, paiements, événements).

La visualisation est assurée via **Grafana** (ou équivalent cloud), avec alertes configurées sur seuils critiques (erreurs, saturation, indisponibilité).

Objectif : détecter de manière proactive toute dérive avant impact majeur utilisateur.



4.7.3 - Traces (option v2 OpenTelemetry/Jaeger)

En cible v2, la traçabilité distribuée est renforcée via **OpenTelemetry + Jaeger**.

Ce dispositif permet de reconstituer les parcours bout en bout entre composants (API, base, cache, messaging, intégrations externes).

Usages attendus :

- Analyse fine des temps de réponse ;
- Localisation rapide des points de contention ;
- Diagnostic multi-composants lors d'incidents complexes.

4.7.4 - Runbook incident et supervision

Un runbook d'exploitation formalise les procédures de gestion d'incident.

Contenu minimal :

- Qualification des alertes (criticité, impact métier) ;
- Etapes de diagnostic initial (logs, métriques, dépendances) ;
- Actions de remédiation standard (redémarrage contrôlé, rollback, isolation de flux) ;
- Escalade vers les rôles concernés ;
- Communication de statut et clôture avec retour d'expérience.

Ce cadre opérationnel garantit une réponse structurée, limite les temps d'indisponibilité et améliore la résilience globale du service.

4.8 - CI/CD et stratégie de livraison

La chaîne CI/CD de **HandlePizza** vise à industrialiser les livraisons, réduire les risques de régression et garantir des déploiements reproductibles entre environnements.

4.8.1 - Pipeline GitHub Actions

Le pipeline d'intégration et de livraison continue est orchestré via **GitHub Actions**.

Il s'exécute à chaque événement clé (push, pull request, promotion de version) et pilote les étapes de validation, packaging et déploiement.



Principes retenus :

- Workflows versionnés dans le dépôt ;
- Exécution automatique et traçable ;
- Séparation des pipelines par environnement (**DEV, TEST/UAT, PROD**) ;
- Utilisation de secrets sécurisés pour les accès techniques ;
- Historisation des exécutions et statuts de livraison.

4.8.2 - Build/test/scan/deploy

Le pipeline suit un enchaînement standard de qualité et de livraison :

- **Build** : compilation et packaging des composants (backend, frontend, mobile selon périmètre).
- **Test** : exécution des tests unitaires, intégration et API.
- **Scan** : contrôles qualité/sécurité (dépendances, vulnérabilités, règles de code).
- **Deploy** : déploiement automatisé vers l'environnement cible avec vérifications post-déploiement.

Objectif : livrer plus vite tout en maintenant un niveau de qualité constant.

4.8.3 - Stratégie de rollback

La stratégie de rollback permet un retour rapide à un état stable en cas d'incident post-déploiement.

Principes retenus :

- Conservation des versions précédentes déployables ;
- Mécanisme de rollback applicatif immédiat ;
- Validation de santé après rollback ;
- Gestion encadrée des migrations de données (compatibilité ascendante/descendante selon scénario) ;
- Procédure documentée dans le runbook d'exploitation.

Objectif : limiter le temps d'impact utilisateur en cas d'échec de livraison.

4.8.4 - Versionning applicatif et DB

DCAC

DCAC.COM

100 boulevard de Paris – 01 23 45 67 89 10 – DCACemail@dcac.com

S.A.R.L. au capital de 1 000,00 € enregistrée au RCS de Xxxx – SIREN 999 999 999 – Code APE : 6202A



Le versionning est aligné entre code applicatif et schéma de données.

Principes retenus :

- Version applicative explicite par release ;
- Scripts de migration base versionnés via **Flyway** ;
- Traçabilité version app / version DB par environnement ;
- Promotion d'artefacts immuables entre **DEV, TEST/UAT et PROD** ;
- Cohérence stricte entre version déployée et état du schéma.

Cette gouvernance garantit des livraisons contrôlées et une évolution maîtrisée du système **HandlePizza**.

4.9 - Intégrations externes en production

Les intégrations externes en production concernent les services tiers critiques pour le fonctionnement métier : paiement et réapprovisionnement.

Elles sont pilotées avec des mécanismes de résilience, de traçabilité et de supervision afin de limiter l'impact des aléas réseau ou des indisponibilités partenaires.

4.9.1 - Système bancaire (timeouts/retry/circuit breaker)

L'intégration bancaire est utilisée pour l'autorisation ou le refus des transactions de paiement.

Principes retenus :

- Appels **HTTPS/REST** sécurisés ;
- Timeouts explicites pour éviter les blocages ;
- Retry contrôlé sur erreurs transitoires ;
- Circuit breaker pour éviter l'effet de cascade en cas d'indisponibilité ;
- Journalisation des tentatives, réponses et erreurs ;
- Mise à jour cohérente du statut de paiement côté métier.

Objectif : sécuriser le parcours de paiement tout en préservant la stabilité applicative.

4.9.2 - Plateforme fournisseur (idempotence/reprise)



L'intégration fournisseur couvre l'émission des commandes de réapprovisionnement et l'intégration des retours de traitement/réception.

Principes retenus :

- Appels **HTTPS/REST** avec contrôle des erreurs ;
- Idempotence sur les opérations sensibles pour éviter les doublons ;
- Mécanismes de reprise sur incident transitoire ;
- Rapprochement réception fournisseur ↔ mise à jour stock ;
- Traçabilité des échanges par identifiant métier.

Objectif : fiabiliser le cycle de réapprovisionnement et garantir la cohérence stock.

4.9.3 - Supervision des appels sortants

Les appels vers systèmes externes sont supervisés en continu afin de détecter rapidement toute dérive.

Éléments supervisés :

- Latence moyenne et maximale des appels ;
- Taux d'échec / timeouts ;
- Fréquence d'ouverture des circuit breakers ;
- Volume de retries et résultat final ;
- Disponibilité perçue des services partenaires.

Les alertes sont corrélées avec les logs applicatifs et les métriques de plateforme pour faciliter le diagnostic et déclencher les procédures runbook adaptées.

Objectif : maintenir un niveau de service stable malgré les dépendances externes.



5 - ARCHITECTURE LOGICIELLE

5.1 - Principes généraux

L'architecture logicielle de **HandlePizza** est conçue pour assurer la maintenabilité du code, la cohérence des règles métier et la capacité d'évolution fonctionnelle du système.

Elle repose sur une structuration modulaire claire, une séparation stricte des responsabilités techniques et des conventions communes de développement.

5.1.1 - Style retenu (*monolithe modulaire*)

Le style logiciel retenu est un monolithe modulaire.

L'application est déployée comme une unité logique, tout en étant organisée en modules métier distincts (authentification, commande, paiement, stock, réapprovisionnement, référentiels, reporting, notification, audit).

Ce choix permet :

- De limiter la complexité de développement et d'exploitation ;
- De garantir la cohérence transactionnelle des traitements critiques ;
- De faciliter les évolutions progressives sans rupture d'architecture ;
- De préserver la lisibilité du système pour les équipes projet.

5.1.2 - Séparation des responsabilités

La solution applique une séparation stricte entre les couches et les rôles techniques :

- **Présentation** : interfaces web/mobile et gestion des interactions utilisateur ;
- **API/Contrôleurs** : exposition des endpoints REST et orchestration des requêtes ;
- **Services métier** : implémentation des règles fonctionnelles ;
- **Accès aux données** : persistance relationnelle et cache/messaging ;
- **Composants transverses** : sécurité, journalisation, gestion d'erreurs, observabilité.

Cette organisation réduit le couplage, améliore la testabilité des composants et facilite l'isolation des anomalies.



5.1.3 - Conventions de codage et de nommage

Le développement suit des conventions homogènes pour garantir qualité et lisibilité :

- Nommage explicite des modules, classes, endpoints et objets d'échange ;
- Conventions **REST** cohérentes (ressources, verbes **HTTP**, versionning) ;
- Séparation claire entre entités de persistance et **DTO** d'API ;
- Centralisation des règles de validation et des erreurs techniques/métier ;
- Format commun de journalisation et de traçabilité ;
- Respect des standards de style et de qualité via outils d'automatisation (**CI/CD**, tests, contrôle qualité).

Ces conventions constituent un cadre partagé entre développement, tests et exploitation pour assurer la pérennité du code **HandlePizza**.

5.2 - Architecture backend

L'architecture backend de **HandlePizza** est structurée autour d'un socle Spring Boot modulaire, organisé en couches logicielles et modules métier indépendants.

Cette structuration garantit la cohérence des règles métier, la maintenabilité du code et la testabilité des traitements.

5.2.1 - Couches logicielles (Controller, Service, Repository, Domain, Security)

Le backend est organisé en couches techniques complémentaires :

- **Controller** : expose les **endpoints API REST**, gère les requêtes/réponses **HTTP** et délègue la logique métier.
- **Service** : implémente les règles fonctionnelles, orchestre les traitements et la coordination entre modules.
- **Repository** : encapsule l'accès aux données (**JPA/Hibernate**) et les opérations de persistance.
- **Domain** : porte les objets métier centraux, leurs invariants et les statuts de référence.
- **Security** : gère l'authentification **JWT**, l'autorisation **RBAC** et les filtres de sécurité transverses.

Ce découpage limite le couplage inter-composants et clarifie les responsabilités de chaque couche.



5.2.2 - Modules métier (auth, commande, paiement, stock, réappro, référentiels, reporting, notification, audit)

L'application est structurée par modules fonctionnels :

- **Auth** : authentification, gestion des tokens, contrôle des accès ;
- **Commande** : cycle de vie des commandes et transitions de statuts ;
- **Paiement** : paiement en ligne/différé et statuts de règlement ;
- **Stock** : niveaux d'ingrédients, disponibilité produit, seuils ;
- **Réapprovisionnement** : commandes fournisseur et réceptions ;
- **Référentiels** : données structurantes (produits, rôles, points de vente, statuts) ;
- **Reporting** : indicateurs d'activité et vues de pilotage ;
- **Notification** : diffusion des événements utiles au suivi ;
- **Audit** : traçabilité métier et journalisation des actions critiques.

Chaque module expose des services internes explicites et reste isolé des détails d'implémentation des autres modules.

5.2.3 - Objets d'échange (DTO, mapping MapStruct, validation Bean Validation)

Les échanges API s'appuient sur des **DTO** dédiés, distincts des entités de persistance.

Le mapping entre objets métier/entités et **DTO** est réalisé via **MapStruct** pour fiabiliser et standardiser les transformations.

La validation des données entrantes repose sur **Bean Validation** (contraintes de format, obligatoires, bornes, cohérence).

Cette approche permet :

- De stabiliser les contrats API ;
- De limiter l'exposition du modèle interne ;
- De contrôler la qualité des données dès l'entrée applicative.

5.2.4 - Transactions et gestion des erreurs



Les traitements critiques sont encadrés par des transactions applicatives afin de garantir la cohérence des écritures (commande, paiement, stock, réapprovisionnement).

La gestion des erreurs suit un modèle homogène :

- Exceptions techniques et métier clairement distinguées ;
- Mapping systématique vers des statuts **HTTP** cohérents ;
- Messages d'erreur exploitables côté client et supervision ;
- Journalisation des incidents avec identifiant de corrélation.

Les appels externes sont protégés par mécanismes de résilience (timeouts, retry, circuit breaker) pour éviter les effets de cascade.

5.2.5 - Structure des packages/sources backend

La structure des sources backend suit une organisation par module puis par couche, par exemple :

- ...auth.controller / service / repository / domain
- ...commande.controller / service / repository / domain
- ...paiement.controller / service / repository / domain
- ...stock..., ...reapro..., ...referentiels..., ...reporting..., ...notification..., ...audit...

packages transverses : security, config, exception, common, infrastructure.

Ce modèle facilite la navigation dans le code, la répartition du travail en équipe et l'évolution contrôlée du backend.

5.3 - Architecture frontend web

L'architecture frontend web de **HandlePizza** est fondée sur une **SPA React/TypeScript** structurée pour favoriser la réutilisabilité des composants, la clarté des parcours utilisateur et la robustesse des échanges avec le backend.

5.3.1 - Organisation SPA (pages, components, routes, state/query)

L'application web est organisée autour de briques fonctionnelles distinctes :

- **Pages** : écrans métier (catalogue, panier, commande, suivi, pilotage) ;



- **Components** : composants UI réutilisables (cards, formulaires, tableaux, statuts) ;
- **Routes** : routage applicatif par profil et périmètre d'accès ;
- **State/query** : gestion des états d'interface et des données distantes.

Le routage est assuré par **React Router** ; la gestion des données API et du cache client par **TanStack Query** ; les formulaires et validations UI par **React Hook Form + Zod**.

5.3.2 - Gestion des appels API et erreurs UI

Les appels API frontend sont centralisés via une couche cliente dédiée pour homogénéiser :

- La construction des requêtes ;
- L'injection du token **JWT** ;
- La gestion des statuts **HTTP** ;
- Le traitement des erreurs techniques et fonctionnelles.

Côté UI, les erreurs sont restituées de façon explicite selon le contexte métier (paiement refusé, produit indisponible, conflit de statut).

Les mécanismes de retry et de rafraîchissement contrôlé des données sont pilotés côté query client pour maintenir une expérience fluide.

5.3.3 - Structure des sources frontend

La structure des sources frontend est organisée par domaines fonctionnels et composants partagés, par exemple :

- src/pages : vues applicatives ;
- src/components : composants UI réutilisables ;
- src/routes : configuration du routage ;
- src/api : clients API et contrats d'échange ;
- src/features : logique métier par domaine ;
- src/hooks : hooks partagés ;
- src/store ou src/query : gestion d'état / cache ;
- src/utils : fonctions utilitaires ;
- src/styles : thèmes et styles globaux.

Cette structuration facilite la maintenabilité et la montée en charge fonctionnelle du frontend.



5.4 - Architecture mobile (Android/iOS)

L'architecture mobile couvre les usages nomades de **HandlePizza** avec une implémentation native par plateforme, alignée sur les mêmes règles métier que les interfaces web.

5.4.1 - Architecture côté mobile (couche UI / domaine / data)

Les applications mobiles suivent un découpage en couches :

- UI : écrans, composants visuels, navigation ;
- Domaine : cas d'usage métier et orchestration des interactions ;
- Data : accès API, cache local et gestion de la persistance locale technique.

Cette séparation améliore la testabilité, la lisibilité du code et l'indépendance des évolutions UI/métier.

5.4.2 - Client API, auth JWT, gestion offline/réseau variable

Le client mobile consomme les API REST sécurisées via **HTTPS**.

Principes retenus :

- Authentification par token **JWT (Bearer)** ;
- Gestion du renouvellement de session (**refresh token**) ;
- Timeouts et retry contrôlé en contexte réseau instable ;
- Synchronisation à reconnexion pour limiter les incohérences ;
- Gestion propre des erreurs utilisateur vs erreurs techniques.

Objectif : garantir des parcours robustes malgré la variabilité de connectivité mobile.

5.4.3 - Structure des sources mobile

La structure des sources est organisée par plateforme et par couche.

Android (**Kotlin + Jetpack Compose, Retrofit/OkHttp**) :

- ui/ (screens, components, navigation),
- domain/ (use cases, models),
- data/ (api, repositories, mappers, local cache),



- core/ (auth, config, utils, error handling).

iOS (**Swift + SwiftUI, URLSession/Alamofire**) :

- UI/ (views, navigation),
- Domain/ (use cases, models),
- Data/ (network, repositories, mapping, persistence),
- Core/ (auth, config, utilities, error handling).

Cette organisation homogène entre **Android** et **iOS** facilite la cohérence fonctionnelle et la maintenance multi-plateforme.

5.5 - Composants transverses

Les composants transverses regroupent les mécanismes techniques communs à l'ensemble des modules applicatifs.

Ils assurent l'homogénéité des comportements non fonctionnels (sécurité, supervision, configuration) sur l'ensemble de la plateforme **HandlePizza**.

5.5.1 - Sécurité applicative

La sécurité applicative est centralisée pour éviter les divergences d'implémentation entre modules.

Principes retenus :

- Authentification par **JWT (access token + refresh token)** ;
- Autorisation **RBAC** par rôle et périmètre métier ;
- Filtres de sécurité communs sur les **endpoints** sensibles ;
- Gestion homogène des erreurs d'authentification/autorisation ;
- Protection des échanges via **HTTPS** et non exposition des secrets dans le code.

5.5.2 - Observabilité applicative (logs, métriques, traces)

L'observabilité applicative est standardisée entre modules backend et interfaces consommatrices.

Principes retenus :

DCAC

DCAC.COM

100 boulevard de Paris – 01 23 45 67 89 10 – DCACemail@dcac.com

S.A.R.L. au capital de 1 000,00 € enregistrée au RCS de Xxxx – SIREN 999 999 999 – Code APE : 6202A



- Logs structurés (format **JSON**) avec identifiants de corrélation ;
- Métriques techniques et métier (latence, erreurs, volumétrie) ;
- Traces de parcours critiques pour diagnostic bout en bout ;
- Tableaux de bord et alertes pour supervision proactive.

Objectif : faciliter la détection, l'analyse et la résolution rapide des incidents.

5.5.3 - Configuration par environnement

La configuration est externalisée et pilotée par environnement (**DEV, TEST/UAT, PROD**).

Principes retenus :

- Paramètres techniques hors code (URLs, secrets, seuils, toggles) ;
- Valeurs spécifiques par environnement sans modification du binaire ;
- Gestion sécurisée des secrets et accès ;
- Traçabilité des changements de configuration ;
- Cohérence entre configuration applicative et stratégie de déploiement.

Cette approche limite les erreurs de promotion et renforce la reproductibilité des déploiements.

5.6 - Règles d'évolution de l'architecture

L'évolution de l'architecture suit des règles explicites pour préserver la stabilité du système tout en permettant l'ajout progressif de fonctionnalités.

5.6.1 - Ajout d'un module métier

Tout nouveau module métier doit respecter le socle architectural existant.

Règles d'ajout :

- Découpage par couches (**controller/service/repository/domain**) ;
- Contrats API explicites et versionnés ;
- Validation des entrées/sorties et mapping **DTO** standardisé ;
- Intégration aux mécanismes transverses (sécurité, logs, supervision, erreurs) ;
- Couverture de tests minimale (unitaires, intégration, API).



5.6.2 - Comptabilité ascendante API

Les évolutions API doivent préserver la compatibilité des consommateurs existants.

Règles principales :

- Versionning explicite des **endpoints** ;
- Non-régression des champs/contrats déjà publiés ;
- Dépréciation progressive avant suppression ;
- Documentation **OpenAPI** synchronisée avec les changements ;
- Communication des impacts de version aux équipes consommatrices.

Objectif : éviter les ruptures de service sur les interfaces web/mobile et les intégrations externes.

5.6.3 - Critères de refactoring / extraction future

Les décisions de refactoring ou d'extraction (vers service isolé) sont pilotées par des critères objectifs :

- Couplage excessif ou complexité fonctionnelle d'un module ;
- Contraintes de scalabilité spécifiques à un domaine ;
- Fréquence de changement élevée nécessitant un cycle autonome ;
- Exigences de disponibilité/performance distinctes ;
- Coûts opérationnels et bénéfices attendus de l'extraction.

Toute évolution structurelle doit être accompagnée d'une analyse d'impact (fonctionnel, technique, exploitation) et d'un plan de migration maîtrisé.



6 - POINTS PARTICULIERS

Cette section regroupe les dispositions techniques transverses et les règles opérationnelles complémentaires nécessaires à la mise en œuvre, à l'exploitation et à l'évolution de **HandlePizza**.

Elle précise notamment les pratiques de journalisation, de configuration, de gestion des environnements, de packaging/livraison et de supervision, afin de garantir la stabilité du service en production et la maîtrise des évolutions.

6.1 - Règles d'évolution de l'architecture

L'évolution de l'architecture technique de **HandlePizza** doit préserver la stabilité du service, la cohérence des règles métier et la capacité d'exploitation.

Toute évolution significative (nouveau module, changement d'interface, évolution de flux externe) est soumise à une analyse d'impact (fonctionnel, technique, sécurité, supervision) et à une validation en environnement de test/recette avant promotion en production.

6.1.1 - Politique de logs (niveaux, format JSON, corrélation)

La journalisation suit une politique commune à tous les composants applicatifs.

Principes retenus :

- Niveaux de logs standardisés (**DEBUG, INFO, WARN, ERROR**) selon le contexte ;
- Format structuré **JSON** pour faciliter l'indexation et la recherche ;
- Présence d'identifiants de corrélation (requête, commande, utilisateur, point de vente) ;
- Distinction claire entre événements techniques et événements métier ;
- Absence d'informations sensibles en clair dans les journaux.

6.1.2 - Rétention et exploitation (CloudWatch, recherche incident)

Les logs sont centralisés dans **CloudWatch** Logs avec une politique de rétention adaptée à l'environnement (**DEV/TEST/PROD**).



Leur exploitation s'appuie sur des filtres, requêtes de recherche et tableaux de bord de supervision.

Principes retenus :

- Rétention courte en **DEV**, intermédiaire en **TEST/UAT**, renforcée en **PROD** ;
- Conservation suffisante pour audit et post-mortem d'incident ;
- Corrélation logs/métriques/alertes pour accélérer l'investigation ;
- Capacité d'extraction des traces pertinentes pour analyse d'incident.

Objectif : disposer d'un historique exploitable sans dérive de volumétrie ni perte d'information critique.

6.1.3 - Données sensibles et masquage

Les données sensibles sont protégées à toutes les étapes de traitement et de journalisation.

Principes retenus :

- Masquage des données personnelles et de paiement dans les logs ;
- Interdiction d'écrire en clair les secrets techniques (tokens, mots de passe, clés API) ;
- Pseudonymisation/anonymisation des jeux de données hors production ;
- Accès aux journaux restreint par rôles (least privilege) ;
- Conformité des pratiques avec les exigences de sécurité et de confidentialité internes.

Objectif : limiter les risques de fuite d'information et garantir la conformité des usages d'exploitation.

6.2 - Configuration et secrets

La configuration de **HandlePizza** est externalisée afin de dissocier le code applicatif des paramètres d'exécution.

Cette approche garantit des déploiements reproductibles, une adaptation par environnement et une meilleure sécurité des informations sensibles.

6.2.1 - Paramètres par environnements (DEV/TEST/PROD)



Les paramètres techniques et fonctionnels sont définis par environnement, sans modification du binaire applicatif.

Principes retenus :

- Fichiers/profils de configuration séparés par environnement ;
- Valeurs adaptées au contexte (URLs, niveaux de logs, seuils, options) ;
- Cohérence de structure entre environnements pour limiter les écarts ;
- Traçabilité des changements de configuration ;
- Validation systématique en **TEST/UAT** avant application en **PROD**.

Objectif : sécuriser la promotion des versions et éviter les dérives de comportement entre environnements.

6.2.2 - Datasources (RDS, pool, timeouts)

La configuration des accès données est centralisée et paramétrée selon la charge attendue.

Éléments configurés :

- Endpoint **RDS MySQL** par environnement ;
- Paramètres de pool de connexions (taille minimale/maximale, idle, lifetime) ;
- Timeouts de connexion, lecture et transaction ;
- Seuils d'alerte liés à la saturation des connexions ;
- Politique de retry contrôlé côté applicatif.

Objectif : garantir la stabilité des accès base, éviter l'épuisement des connexions et maintenir les performances.

6.2.3 - Secrets Manager (rotation, accès)

Les secrets techniques sont stockés hors code dans **AWS Secrets Manager**.

Principes retenus :

- Stockage centralisé des identifiants sensibles (DB, API externes, clés applicatives) ;
- Récupération dynamique au runtime ;
- Rotation des secrets selon politique de sécurité ;
- Contrôle d'accès **IAM** au plus juste privilège ;



- Audit des accès et des modifications de secrets.

Objectif : réduire le risque d'exposition des secrets et renforcer la sécurité opérationnelle.

6.2.4 - Feature flags / paramètres métiers

Les paramètres métier évolutifs sont pilotés sans redéploiement complet lorsque possible.

Exemples de paramètres :

- Activation progressive de fonctionnalités ;
- Seuils et règles non structurales ;
- Options de comportement selon contexte opérationnel.

Principes retenus :

- Pilotage centralisé et versionné des flags/paramètres ;
- Activation contrôlée par environnement ;
- Journalisation des changements ;
- Rollback rapide en cas d'effet indésirable.

Objectif : permettre des évolutions incrémentales maîtrisées et réduire le risque en production.

6.3 - Ressources et performances

Cette section précise les hypothèses de capacité de la plateforme, les limites techniques connues et les objectifs de performance à tenir pour garantir une exploitation stable de **HandlePizza**.

6.3.1 - Dimensionnement cible (API, DB, cache, MQ)

Le dimensionnement cible est défini par couche, avec ajustement progressif selon la charge réelle observée.

Principes de dimensionnement :

- **API backend** : nombre minimal de tâches en régime nominal et montée en charge horizontale automatique lors des pics ;



- **Base de données (RDS MySQL)** : capacité dimensionnée pour absorber les opérations transactionnelles critiques (commande, paiement, stock) ;
- **Cache (Redis)** : mémoire allouée pour les données fréquemment consultées (catalogue, disponibilité, référentiels) ;
- **Messaging (RabbitMQ)** : files dimensionnées pour lisser les traitements asynchrones sans bloquer le parcours utilisateur.

Le dimensionnement initial est validé en test de charge puis ajusté via métriques de production (CPU, mémoire, latence, files d'attente).

6.3.2 - Limites techniques connues

Les limites techniques identifiées à ce stade sont les suivantes :

- Dépendance à des services tiers (banque/fournisseur) avec variabilité de disponibilité ;
- Sensibilité aux pics de trafic simultanés sur les parcours de commande/paiement ;
- Saturation possible des connexions DB en cas de mauvais paramétrage de pool ;
- Dérive de latence si les files asynchrones ne sont pas correctement supervisées ;
- Impact potentiel des migrations DB si fenêtre de déploiement insuffisamment maîtrisée.

Ces limites sont adressées par des mécanismes de résilience, de supervision et de montée en charge progressive.

6.3.3 - Objectifs de performance (latence, débit, charge)

Les objectifs de performance visent à maintenir une expérience fluide pour les utilisateurs finaux et les équipes opérationnelles.

Objectifs cibles :

- **Latence API** : temps de réponse court sur opérations critiques (commande, statut, paiement) ;
- **Débit** : capacité à absorber des pics de requêtes sans dégradation majeure ;
- **Charge** : maintien de la stabilité applicative en période de rush ;
- **Disponibilité** : continuité de service avec reprise automatique en cas d'incident local ;
- **Traitements asynchrones** : exécution différée sans impact visible sur la navigation temps réel.



Les seuils précis de **SLO/SLA** sont définis en exploitation et suivis via les outils d'observabilité.

6.4 - Environnement de développement

L'environnement de développement local est standardisé pour garantir la reproductibilité des builds, limiter les écarts entre postes et faciliter l'onboarding des contributeurs.

6.4.1 - Prérequis poste de développement

Chaque poste de développement doit disposer des outils suivants :

- **Git** (gestion de versions) ;
- **JDK 17** (backend **Spring Boot**) ;
- **Node.js LTS** + **npm/pnpm** (frontend **React**) ;
- **Docker Desktop** (services locaux DB/cache/messaging) ;
- **IDE : IntelliJ IDEA / VS Code / Android Studio / Xcode** selon périmètre ;
- Accès aux dépôts, registres et secrets non sensibles de développement.

Prérequis système recommandés :

- OS à jour (Windows/macOS/Linux),
- Mémoire suffisante pour exécuter backend + frontend + services conteneurisés,
- Connectivité stable pour dépendances et intégrations de test.

6.4.2 - Lancement local (backend, frontend, DB, MQ)

Le lancement local suit une séquence standard :

- Démarrer les services techniques locaux (**MySQL, Redis, RabbitMQ**) via **Docker Compose** ;
- Appliquer les migrations de schéma (**Flyway**) ;
- Lancer le backend **Spring Boot** avec profil dev ;
- Lancer le frontend web (serveur **Vite**) ;
- Vérifier les endpoints de santé et la documentation **OpenAPI/Swagger**.

En mode mobile, les applications Android/iOS pointent vers les endpoints de l'environnement local ou DEV selon le scénario de test.



6.4.3 - Qualité locale (tests, lint, format)

Avant toute intégration, les contrôles qualité locaux sont obligatoires :

- Exécution des tests unitaires backend/frontend ;
- Exécution des tests d'intégration ciblés (si modification impactante) ;
- Contrôle de linting et formatage (code style) ;
- Vérification des contrats API et non-régression fonctionnelle minimale.

Objectif : détecter les anomalies en amont et réduire les échecs dans la pipeline **CI/CD**.

6.5 - Packaging, livraison et déploiement

La stratégie de packaging et de livraison de **HandlePizza** vise à garantir des déploiements reproductibles, traçables et sûrs entre les environnements DEV, TEST/UAT et PROD.

6.5.1 - Build artefacts (backend/frontend/mobile)

Chaque composant produit un artefact versionné, prêt à être promu sans reconstruction.

Artefacts principaux :

- **Backend** : image conteneurisée **Spring Boot (Java 17)** ;
- **Frontend web** : bundle statique optimisé (assets déployables **S3/CloudFront**) ;
- **Mobile** : artefacts applicatifs Android/iOS selon le cycle de publication interne.

Principes retenus :

- Artefacts immuables ;
- Version unique par release ;
- Traçabilité entre commit source, build et artefact produit.

6.5.2 - Pipeline CI/CD (étapes et controles)

Le pipeline **CI/CD** orchestre les étapes de qualité et de promotion :

- Checkout et préparation de l'environnement d'exécution ;
- Build des composants ;
- Exécution des tests (unitaires, intégration, API) ;



- Contrôles qualité/sécurité (lint, dépendances, vulnérabilités) ;
- Publication des artefacts ;
- Déploiement vers l'environnement cible ;
- Vérifications post-déploiement (santé, logs, métriques).

Objectif : automatiser les livraisons tout en maintenant un niveau de qualité constant.

6.5.3 - Stratégie de rollback

La stratégie de rollback permet un retour rapide à une version stable en cas d'incident.

Principes retenus :

- Conservation des versions précédentes déployables ;
- Rollback applicatif contrôlé ;
- Vérification automatique des endpoints de santé après retour arrière ;
- Coordination avec les changements de schéma DB ;
- Documentation des procédures dans le runbook d'exploitation.

Objectif : minimiser l'impact utilisateur en cas d'échec de déploiement.

6.5.4 - Versionnage app + DB (Flyway)

Le versionnage applicatif et le versionnage base de données sont synchronisés.

Principes retenus :

- Version applicative explicite par release ;
- Scripts **Flyway** versionnés et ordonnés ;
- Exécution des migrations dans la chaîne **CI/CD** selon environnement ;
- Compatibilité contrôlée entre version applicative et schéma ;
- Traçabilité des migrations exécutées par environnement.

Objectif : garantir une évolution cohérente du code et des données, sans rupture de service.

6.6 - Exploitation et continuité

L'exploitation de **HandlePizza** est organisée pour garantir la disponibilité du service, la continuité d'activité et la capacité de reprise en cas d'incident.



Elle s'appuie sur une supervision active, des procédures de sauvegarde/restauration et un cadre opérationnel documenté.

6.6.1 - Supervision & alerting

La supervision couvre l'ensemble des couches techniques (exposition, backend, données, messaging, intégrations externes).

Les alertes sont configurées sur les indicateurs critiques :

- Disponibilité des services ;
- Latence API ;
- Taux d'erreurs ;
- Saturation ressources (CPU, mémoire, connexions DB, files MQ) ;
- Dérives d'intégrations externes.

Objectif : détecter rapidement les anomalies et déclencher une réaction proportionnée à la criticité.

6.6.2 - Sauvegardes/restauration

La continuité des données repose sur une politique de sauvegardes planifiées et testées.

Principes retenus :

- Sauvegardes automatiques de la base **MySQL** avec rétention définie ;
- Protection et chiffrement des sauvegardes ;
- Tests périodiques de restauration ;
- Validation de l'intégrité des données restaurées ;
- Documentation des fenêtres et procédures de reprise.

Objectif : garantir la récupération des données métier dans un délai maîtrisé.

6.6.3 - Runbook incident

Le runbook formalise les procédures de gestion d'incident et les rôles associés.

Contenu minimal :

DCAC

DCAC.COM

100 boulevard de Paris – 01 23 45 67 89 10 – DCACemail@dcac.com

S.A.R.L. au capital de 1 000,00 € enregistrée au RCS de Xxxx – SIREN 999 999 999 – Code APE : 6202A



- Qualification de l'incident (impact, criticité, périmètre) ;
- Etapes de diagnostic (logs, métriques, traces, dépendances externes) ;
- Actions de remédiation standard (redémarrage, rollback, isolation d'un flux) ;
- Escalade vers les responsables techniques/métier ;
- Communication de statut et clôture avec retour d'expérience.

Objectif : réduire le temps de résolution et homogénéiser la réponse opérationnelle.

6.6.4 - Plan d'évolution (v2)

Le plan d'évolution v2 permet d'anticiper les besoins futurs de montée en charge et de maturité technique.

Axes envisagés :

- Renforcement de la traçabilité distribuée (**OpenTelemetry/Jaeger**) ;
- Optimisation progressive des performances et du dimensionnement ;
- Amélioration des mécanismes de résilience sur intégrations externes ;
- Extension des dashboards métiers et techniques ;
- Evolution modulaire guidée par la charge et le couplage des composants.

Objectif : accompagner la croissance du réseau **OC Pizza** sans rupture de service ni dette technique non maîtrisée.



7 - GLOSSAIRE

ACM (AWS Certificate Manager)	Service AWS qui gère les certificats TLS/SSL. Il évite de gérer manuellement la création, le renouvellement et l'installation des certificats utilisés pour sécuriser les accès HTTPS.
ALB (application load balancer)	Répartiteur de charge AWS pour le trafic HTTP/HTTPS. Il distribue les requêtes entrantes vers les instances backend disponibles et améliore disponibilité et montée en charge.
API (application programming interface)	Interface d'échange entre composants logiciels. Dans HandlePizza , les API REST exposent les fonctions métier (commande, paiement, stock, etc.).
Artefact	Livrable technique produit par le pipeline (image conteneur, bundle frontend, package mobile), versionné et promu entre environnements sans reconstruction.
Audit trail	Journal horodaté des actions critiques (qui a fait quoi, quand, sur quel objet métier), utilisé pour la traçabilité, l'investigation et la conformité.
Auto scaling horizontal	Mécanisme qui augmente/réduit automatiquement le nombre d'instances backend selon la charge, pour maintenir les performances en période de rush.
Backend	Partie serveur de l'application qui contient la logique métier, les règles de sécurité et l'accès aux données.
Bean Validation	Standard Java de validation des données (formats, champs obligatoires, bornes). Il garantit la qualité des entrées API avant traitement métier.
Build	Étape de compilation et de packaging du code source en artefacts déployables.
Cache	Zone mémoire rapide utilisée pour éviter de recalculer/recharger des données fréquemment consultées, et réduire la latence.
CI/CD	Chaîne d'intégration et livraison/déploiement continu. Elle automatise build, tests, contrôles qualité, puis déploiement.
Circuit breaker	Mécanisme de résilience qui interrompt temporairement les appels vers un service externe en panne pour éviter l'effet domino.
CloudFront	CDN (Content delivery Network) AWS qui distribue les assets frontend



	(HTML/CSS/JS, images) depuis des points proches des utilisateurs pour réduire la latence.
CloudWatch	Service AWS de supervision centralisée (logs, métriques, alertes) pour détecter et diagnostiquer les incidents.
Container	Unité d'exécution isolée contenant l'application et ses dépendances, facilitant déploiements homogènes et reproductibles.
Controller	Couche backend recevant les requêtes HTTP, appliquant les règles d'entrée et déléguant le traitement aux services métier.
Corrélation (ID de corrélation)	Identifiant unique attaché à un flux pour relier logs, métriques et événements d'une même opération.
Datasource	Configuration d'accès à la base de données (URL, identifiants, pool, timeouts).
DTO (Data Transfer Object)	Objet d'échange API, distinct des entités DB, pour stabiliser le contrat externe et éviter d'exposer le modèle interne.
ECS fargate	Service AWS d'exécution de conteneurs sans gestion directe de serveurs, adapté aux déploiements backend scalables.
Endpoint	URL d'une ressource API (ex: /api/v1/commands/{id}).
Feature flag	Interrupteur de fonctionnalité activable/désactivable sans redéploiement, utile pour déploiements progressifs et rollback rapide.
Flyway	Outil de migration/versionnage du schéma DB. Il applique les scripts SQL dans le bon ordre et garde l'historique par environnement.
Frontend	Partie interface utilisateur (web/mobile) qui consomme les API et présente les parcours métier.
Grafana	Outil de dashboards et visualisation de métriques, utilisé pour suivre la santé et les performances.
Health check	Test automatisé de disponibilité d'un service. Il permet d'exclure du routage les instances non saines.
HTTPS	HTTP chiffré via TLS, garantissant confidentialité et intégrité des échanges.
IAM (Identity and Access Management)	Mécanisme AWS de gestion des identités et permissions, pour appliquer le moindre privilège.
Idempotence	Propriété d'une opération qui donne le même résultat si répétée



	plusieurs fois, essentielle pour éviter les doublons lors de retries.
JDBC	Couche Java de connexion/exécution SQL vers une base relationnelle.
JPA (Java Persistence API)	Standard Java d'accès objet-relationnel, utilisé avec Hibernate pour persister/requêter les entités métier.
JWT (JSON Web Token)	Jeton signé transportant l'identité et les droits d'un utilisateur pour sécuriser les API.
Logback JSON	Format de logs structurés JSON facilitant indexation, recherche et corrélation en supervision.
MapStruct	Librairie de mapping automatique entre objets (entités ↔ DTO), réduisant le code manuel et les erreurs de transformation.
Messaging	Echanges asynchrones via files de messages pour découpler producteurs/consommateurs et lisser la charge.
Micrometer	Librairie de collecte de métriques applicatives (latence, erreurs, compteurs), exportables vers Prometheus/Cloud./consommateurs et lisser la charge.
Monolithe modulaire	Architecture déployée en une application unique, mais organisée en modules métier séparés et clairement découplés.
MQ (Message Queue)	File de messages entre composants pour traitement différé et robuste.
NAT Gateway	Service réseau permettant aux services privés d'initier des sorties Internet sans être directement exposés.
Nginx	Serveur web/reverse proxy, utilisé comme point d'entrée HTTPS et routage vers backend.
OpenAPI / Swagger	Standard et outils de documentation des API (contrats, schémas, statuts), utiles pour développement, test et maintenance.
OpenTelemetry	Standard de collecte de traces/métriques/logs pour observabilité distribuée.
RBAC (Role-Based Access Control)	Autorisation basée sur les rôles (client, employé, manager, etc.) et les périmètres.
RDS MySQL	Base de données MySQL managée AWS, avec sauvegardes, supervision et fonctionnalités de restauration.



Redis	Base clé-valeur en mémoire utilisée principalement pour le cache à faible latence.
Repository	Couche backend dédiée aux accès données, isolant la logique de persistance du reste du métier.
Resilience4j	Bibliothèque de résilience Java (retry, circuit breaker, bulkhead, timeout) pour sécuriser les appels externes.
REST	Style d'API basé sur ressources HTTP, verbes standards et statuts normalisés.
Retry	Nouvelle tentative automatique après échec transitoire (réseau/timeout), encadrée pour éviter surcharge.
Rollback	Retour à une version précédente stable suite à incident de déploiement.
Runbook	Procédure opérationnelle documentée pour diagnostiquer et traiter les incidents.
S3 (Amazon Simple Storage Service)	Stockage objet AWS utilisé pour héberger les assets frontend statiques.
Scheduler	Mécanisme exécutant des tâches planifiées (nettoyage, synchronisation, traitements périodiques).
Secrets Manager	Service AWS de stockage sécurisé des secrets (mots de passe DB, clés API), avec rotation et audit.
Security Group	Pare-feu AWS au niveau ressource, autorisant uniquement les flux réseau nécessaires.
SLA (Service Level Agreement)	Engagement contractuel de niveau de service (disponibilité/performance).
SLO (Service level Objective)	Objectif interne chiffré de performance/disponibilité suivi opérationnellement.
SPA (Single Page Application)	Application web à page unique, navigation dynamique sans rechargement complet.
Spring Boot	Framework Java pour construire des API backend rapidement avec conventions robustes.
Spring Security	Composant Spring gérant authentification, autorisation, filtres et protection des endpoints.
SSE (Server-Sent Events)	Mécanisme de push serveur→client en HTTP, adapté au temps réel léger (statuts).



Subnet (sous-réseau)	Segment réseau isolé dans un VPC (public/privé), utilisé pour contrôler l'exposition des services.
TLS (Transport Layer Security)	Protocole cryptographique protégeant les communications réseau.
UAT (User Acceptance Cloud)	Environnement/phase de recette utilisateur pour valider la conformité métier avant PROD.
VPC (Virtual Private Cloud)	Réseau privé virtuel AWS isolant les ressources cloud du projet.
WebSocket(STOMP)	Protocole cryptographique protégeant les communications réseau.